

mods/clients/isisclient.cpp

[Go to the documentation of this file.](#)

```
1  /*!  
2  \brief ISIS UDP socket client  
3  
4  This class defines an instance of ISIS UDP socket client. It  
5  provides data and methods for sending and receiving IMPv2 messages  
6  to and from an ISIS server and other ISIS nodes. It also provides a  
7  method for parsing message body contents into a hash table by  
8  keyword=value pairs, but otherwise it does no interpretation of the  
9  incoming message beyond the most basic task of identifying the  
10 message type and teasing out its component parts.  
11  
12 There are three ways to send commands:  
13     1) Immediate - send it directly to the socket and don't wait  
14     2) Send-and-Wait - send immediately and wait for a reply (block)  
15     3) Queue - queue the message to the dispatcher thread (non-blocking)  
16 The dispatcher thread is a simple message dispatcher queue that will  
17 dispatch each message at a given cadence.  
18  
19 */  
20  
21 #include <QtGui>  
22 #include <QtNetwork>  
23  
24 #include "isisclient.h"  
25  
26 ISISClient::ISISClient(const QString &serverID,  
27                        const QString &serverHost,  
28                        const int &serverPort,  
29                        const QString &myID,  
30                        const int &myPort,  
31                        QWidget *parent)  
32 : QWidget(parent)  
33 {  
34  
35     // ISIS server host address, port, and ID  
36  
37     isisServerHost = serverHost;  
38     if (serverHost.compare("localhost",Qt::CaseInsensitive)==0)  
39         isisServerAddr = new QHostAddress(QHostAddress::LocalHost);  
40     else  
41         isisServerAddr = new QHostAddress(serverHost);  
42  
43     isisServerPort = serverPort;  
44     isisServerID = serverID;  
45  
46     // This client's ID and socket port (localhost is implicit)  
47  
48     clientID = myID;  
49     clientPort = myPort;  
50  
51     // Basic initializations  
52  
53     datagramStr.clear();  
54     errorStr.clear();  
55     senderID.clear();  
56     isConnected = false; // socket not yet bound to the client port  
57     udpSocket = 0;  
58  
59     // Setup but do not yet enable the heartbeat timer and its defaults  
60  
61     heartbeatTimer = new QTimer(this);  
62     connect(heartbeatTimer,SIGNAL(timeout()),this,SLOT(sendHeartBeat()));  
63     heartbeat = false; // disable the heartbeat by default  
64     heartbeatTime = 120; // default heartbeat time is 120 seconds  
65  
66 }  
67  
68 //  
69 // init() - create the socket and bind it to the client port  
70 //  
71 // This is our success/fail point in establishing the client socket.  
72 // Also connects the readyRead() signal to the readDatagram() slot,
```

```

73 // and sets up the dispatcher queue and the heartbeat timer.
74 //
75
76 bool ISISClient::init()
77 {
78     // Create the client socket
79
80     if (!udpSocket)
81         udpSocket = new QUdpSocket(this);
82
83     // Bind this client socket to clientPort, where clientPort=0 is
84     // interpreted as "any free port" on localhost.
85
86     if (!udpSocket->bind(clientPort)) {
87         errorStr = "Could not bind client socket on port " +
88             QString::number(clientPort) + " - " + udpSocket->errorString();
89         isConnected = false;
90         return false;
91     }
92     isConnected = true;
93
94     // Connect the readyRead() signal to the getDatagram() slot
95
96     connect(udpSocket, SIGNAL(readyRead()), this, SLOT(readDatagram()));
97
98     // Connect the command sequencer thread's dispatch() signal to
99     // our sendDispatch() slot, and set the cadence to the default value
100     // of 1 second (1000 msec)
101
102     msgQueue.setCadence(1000);
103     connect(&msgQueue, SIGNAL(dispatch(const QString &, const QString &)),
104         this, SLOT(sendDispatch(const QString &, const QString &)));
105
106     // If heartbeat is enabled, start the heartbeat timer
107
108     if (heartbeat) enableHeartBeat(true);
109
110     return true;
111 }
112
113 //
114 // isReady() - is the socket initialized and ready to use?
115 //
116
117 bool ISISClient::isReady()
118 {
119     return isConnected;
120 }
121
122 //
123 // send() - Send a datagram message string in IMPv2 protocol format
124 //
125
126 bool ISISClient::send(const QString &toAddr, const QString &msgStr)
127 {
128     // Do not send unaddressed messages...
129
130     if (toAddr.isEmpty()) {
131         errorStr = "Destination address is empty, nobody to send a message to";
132         return false;
133     }
134
135     // We can't send messages unless we've initialized the client UDP socket
136
137     if (!isConnected) {
138         errorStr = "No Client socket initialized";
139         return false;
140     }
141
142     // We have something to send and an initialized socket, go for it
143
144     QByteArray datagram = "";
145
146     // Build the IMPv2 datagram...
147
148     if (msgStr.isEmpty()) { // heartbeat pulse
149         datagram.append(QString("%1>%2\r").arg(clientID).arg(toAddr.toUpper()));
150     }
151     else {

```

```

152     datagram.append(QString("%1>%2 %3\r").arg(clientID).arg(toAddr.toUpper()).arg
(msgStr));
153     }
154
155     // ...and send it
156
157     udpSocket->writeDatagram(datagram.data(),
158                             datagram.size(),
159                             *isisServerAddr,
160                             isisServerPort);
161
162     // log message signal
163
164     emit logMessage(QString::QString(datagram).trimmed());
165
166     // all done!
167
168     return true;
169
170 }
171
172 //
173 // sendAndWait() - Send a datagram string to the ISIS server
174 //                  and wait for the readyRead signal or timeout
175 //
176
177 bool ISISClient::sendAndWait(const QString &toAddr, const QString &msgStr,
178                             const int &timeout)
179 {
180     // Send the datagram via our send() method...
181
182     if (!send(toAddr,msgStr)) return false;
183
184     // ...and wait up to timeout milliseconds for a ReadyRead signal.
185
186     if (timeout>0) {
187         QApplication->processEvents();
188         if (!udpSocket->waitForReadyRead(timeout)) {
189             errorStr = "Timeout on datagram socket reply wait";
190             return false;
191         }
192     }
193     errorStr.clear();
194     return true;
195 }
196
197 //
198 // readDatagram() slot - read a pending datagram from the socket
199 //
200 // This is invoked by the udpSocket object's readyRead() signal. It
201 // reads the pending datagram from the client socket, sets the
202 // datagramStr datum, and emits the client's dataReady signal.
203 //
204 // It also parses the datagram into components and validates it,
205 // making these data available through the various methods in this
206 // class provided for that purpose, though a user can elect to
207 // separately parse the entire message independently.
208 //
209
210 void ISISClient::readDatagram()
211 {
212     datagramStr.clear();
213
214     if (udpSocket->hasPendingDatagrams()) {
215         QByteArray datagram;
216
217         datagram.resize(udpSocket->pendingDatagramSize());
218         udpSocket->readDatagram(datagram.data(),datagram.size());
219
220         datagramStr.append(datagram);
221
222         // Strip the datagram of extraneous whitespace & terminators
223
224         datagramStr = datagramStr.trimmed();
225
226     }
227
228     parse(datagramStr); // parse it into components and validate it
229

```

```

230     emit dataReady();
231     emit logMessage(datagramStr);
232 }
233
234 //
235 // text() - Return the full text of the most recent datagram text
236 //
237
238 QString ISISClient::text()
239 {
240     return datagramStr;
241 }
242
243 //
244 // parse() - Parse an IMPv2 message into components, with syntax validation
245 //
246 // An IMPv2 protocol message has the following structure
247 //
248 //     fromID>destID msgType: body of the message\r
249 //
250 // where:
251 //     fromID   = ID of the sender
252 //     destID   = ID of the recipient
253 //     msgType  = type of message, one of REQ:, EXEC:, DONE:, STATUS:, WARNING:,
254 //               ERROR:, or FATAL:
255 //     followed by the body of the message, the first word of which is the
256 //     command string, except for ping/pong and broadcast messages.
257 //
258 // Note that if there is no message type it is interpreted as an
259 // implicit REQ: (command request) message.
260 //
261
262 bool ISISClient::parse(const QString &msgStr)
263 {
264     // Initializations
265
266     messageType = UNKNOWN;
267     messageBody.clear();
268     senderID.clear();
269     recipientID.clear();
270     isIMPv2 = false;
271
272     // Message addressing (routing) token
273
274     QString msgAddr = msgStr.section(" ",0,0);
275
276     // Does the address token have a >? This is our format validator
277
278     if (msgAddr.contains(">",Qt::CaseInsensitive)) {
279         senderID = msgAddr.section(">",0,0);
280         recipientID = msgAddr.section(">",1,1);
281         isIMPv2 = true;
282     }
283     else {
284         errorStr = "Message does not have valid IMPv2 format";
285         isIMPv2 = false;
286         return false;
287     }
288
289     // We have a valid message, determine the message type and extract
290     // the body of the message
291
292     QString typeStr = msgStr.section(" ",1,1); // Message type token
293     messageBody = msgStr.section(" ",2,-1); // Body of the message
294
295     if (typeStr.compare("ERROR:",Qt::CaseInsensitive)==0)
296         messageType = ERROR;
297     else if (typeStr.compare("WARNING:",Qt::CaseInsensitive)==0)
298         messageType = WARNING;
299     else if (typeStr.compare("FATAL:",Qt::CaseInsensitive)==0)
300         messageType = FATAL;
301     else if (typeStr.compare("DONE:",Qt::CaseInsensitive)==0)
302         messageType = DONE;
303     else if (typeStr.compare("STATUS:",Qt::CaseInsensitive)==0)
304         messageType = STATUS;
305     else if (typeStr.compare("EXEC:",Qt::CaseInsensitive)==0)
306         messageType = EXEC;
307     else if (typeStr.compare("REQ:",Qt::CaseInsensitive)==0)
308         messageType = REQ;

```

```

309     else { // implicit REQ:
310         messageType = REQ;
311         messageBody = msgStr.section(" ",1,-1);    // Body of the message
312     }
313
314     return true;
315 }
316
317 //
318 // from() - Return the sender ID of the most recent datagram
319 //
320 //
321
322 QString ISISClient::sender()
323 {
324     return senderID;
325 }
326
327 //
328 // dest() - Return the destination ID of the most recent datagram
329 //
330
331 QString ISISClient::recipient()
332 {
333     return recipientID;
334 }
335
336 //
337 // type() - Return the message type code of the most recent datagra
338 //
339
340 int ISISClient::type()
341 {
342     return messageType;
343 }
344
345 //
346 // body() - Return the message body of the most recent datagra
347 //
348
349 QString ISISClient::body()
350 {
351     return messageBody;
352 }
353
354 //
355 // isValid() - Was the most recent datagram a valid IMPv2 message
356 //
357
358 bool ISISClient::isValid()
359 {
360     return isIMPv2;
361 }
362
363 //
364 // setClient() - Set the client ID (IMPv2 node name) and port
365 //
366
367 void ISISClient::setClient(const QString &idStr, const int &portNum)
368 {
369     clientID = idStr;
370     clientPort = portNum;
371 }
372
373 // id() - Return the client ID (IMPv2 node name)
374
375 QString ISISClient::id()
376 {
377     return clientID;
378 }
379
380 // port() - Return the client port number
381
382 int ISISClient::port()
383 {
384     return clientPort;
385 }
386
387 //

```

```

388 // setServer() - Set the ISIS server host and port settings
389 //
390
391 void ISISClient::setServer(const QString &idStr, const QString &hostStr,
392                           const int &portNum)
393 {
394     isisServerID = idStr;
395     isisServerHost = hostStr;
396     if (isisServerHost.compare("localhost", Qt::CaseInsensitive)==0)
397         isisServerAddr = new QHostAddress(QHostAddress::LocalHost);
398     else
399         isisServerAddr = new QHostAddress(isisServerHost);
400     isisServerPort = portNum;
401 }
402
403 //
404 // isisID() - Return the ISIS server ID (IPv2 node name)
405 //
406
407 QString ISISClient::isisID()
408 {
409     return isisServerID;
410 }
411
412 //
413 // isisHost() - Return the ISIS server hostname
414 //
415
416 QString ISISClient::isisHost()
417 {
418     return isisServerHost;
419 }
420
421 //
422 // isisPort() - Return the ISIS server port number
423 //
424
425 int ISISClient::isisPort()
426 {
427     return isisServerPort;
428 }
429
430 //
431 // errorString() - Return the most recent error string
432 //
433
434 QString ISISClient::errorString()
435 {
436     return errorStr;
437 }
438
439 //
440 // getKeys - Extract key=value pairs from an IPv2 message string and
441 //           load them into keyword dictionary hash table.
442 //
443 // IPv2 format strings can pass keyword=value and +/- boolean flags
444 // as follows:
445 //
446 //     integer:  keyword=123
447 //     double:   keyword=1.234
448 //              keyword=1.234E+05
449 //     string:   keyword=abcdefg
450 //              keyword='a string of words separated by spaces'
451 //              keyword=(a string of words separated by spaces) <-- old-style
452 //     boolean:  keyword=T
453 //              +keyword (same as keyword=T) <-- old-style
454 //              -keyword (same as keyword=F) <-- old-style
455 //     list:     keyword=(1,2,3,4)
456 //
457 // Returns the number of keys found in the msgStr. Note that it does
458 // not clear the hash table it is given as the container for the
459 // keyword dictionary it extracts, so that this hash can be used to
460 // accumulate a keyword dictionary for an entire session if needed.
461 // How to handle the hash table is left to the calling program - we
462 // just fill the container they provide a pointer to.
463 //
464
465 int ISISClient::getKeys(const QString &msgStr, QHash<QString,QString> *hash)
466 {

```

```

467 // We work with a copy of the string, aggressively stripping it of
468 // extraneous white space, tabs, terminators, etc. to make processing
469 // into keywords simpler.
470
471 QString testStr = msgStr;
472 testStr.simplified();
473
474 QStringList argList = testStr.split(" ");
475 if (argList.size() == 0) return 0;
476
477 // We have at least 1 word to check...
478
479 bool quoted = false;
480 QString closeQuote;
481 int numKeys = 0;
482 int i = 0;
483
484 while (i < argList.size()) {
485     // Is this argument a keyword=value pair?
486
487     if (argList.at(i).contains("=", Qt::CaseInsensitive)) {
488         quoted = false;
489         QString newKey = argList.at(i).section("=", 0, 0);
490         QString newVal = argList.at(i).section("=", 1, -1);
491
492         // Look for a single quote or ( in the first character of newVal
493         // to see if this is a multi-word string value
494
495         if (newVal.startsWith("'")) { // single-quote delimited string value
496             if (newVal.endsWith("'"))
497                 quoted = false; // single word inside single quotes
498             else {
499                 quoted = true;
500                 closeQuote = "'";
501             }
502             newVal.remove("'");
503         }
504         else if (newVal.startsWith("(")) { // () delimited string value
505             if (newVal.endsWith("(")) {
506                 quoted = false; // single word inside ()s
507                 newVal.remove("(");
508             }
509             else {
510                 quoted = true;
511                 closeQuote = ")";
512             }
513             newVal.remove("(");
514         }
515         else { // single-word value
516             quoted = false;
517         }
518
519         // If a quoted multi-word string value, build it up until we
520         // find the closing quote character
521
522         while (quoted) {
523             i++;
524             if (i < argList.size()) {
525                 QString nextStr = argList.at(i);
526                 if (nextStr.endsWith(closeQuote)) { // closing quote found
527                     nextStr.remove(closeQuote);
528                     quoted = false;
529                 }
530                 newVal.append(" ");
531                 newVal.append(nextStr);
532             }
533             else
534                 quoted = false; // premature end of value
535         }
536
537         // Add the keyword and its value to the hash
538
539         hash->insert(newKey, newVal);
540         numKeys++;
541     }
542     i++;
543 }
544
545 // Is this argument a +key boolean?

```

```

546     else if (argList.at(i).size()>1 && argList.at(i).startsWith("+") ){
547         QString newKey = argList.at(i);
548         newKey.remove("+");
549         hash->insert(newKey, "T");
550         numKeys++;
551     }
552
553     // Is this argument a -key boolean?
554
555     else if (argList.at(i).size()>1 && argList.at(i).startsWith("-")) {
556         QString newKey = argList.at(i);
557         newKey.remove("-");
558         hash->insert(newKey, "F");
559         numKeys++;
560     }
561
562     i++; // increment the argument counter
563 }
564
565 // All done, return the number of keys we found
566
567 return numKeys;
568
569 }
570
571 // Queue a message to dispatcher thread
572
573 void ISISClient::queue(const QString &host, const QString &body)
574 {
575     if (host.isEmpty()) return;
576     if (body.isEmpty()) return;
577     msgQueue.addMessage(host, body);
578 }
579
580 // Send a message from the dispatcher queue (implemented as a private slot)
581
582 void ISISClient::sendDispatch(const QString &host, const QString &body)
583 {
584     if (!send(host, body)) emit error();
585 }
586
587 // Set the message dispatcher queue cadence in milliseconds
588
589 void ISISClient::setDispatchCadence(const int &cadence)
590 {
591     if (cadence > 0)
592         msgQueue.setCadence(cadence);
593 }
594
595 // Abort the message dispatcher queue - implemented as a public slot
596
597 void ISISClient::abortQueue()
598 {
599     msgQueue.abort();
600 }
601
602 // Enable/Disable the heartbeat
603
604 void ISISClient::enableHeartBeat(const bool &hbState)
605 {
606     if (hbState) {
607         heartBeatTimer->stop(); // just in case
608         if (heartBeatTime > 0) {
609             heartBeat = true;
610             heartBeatTimer->start(heartBeatTime*1000);
611         }
612     }
613     else {
614         heartBeatTimer->stop();
615         heartBeat = false;
616     }
617 }
618
619 // Set the heartBeat time interval in seconds. If set to zero, it also
620 // disables the heartbeat timer
621
622 void ISISClient::setHeartBeatTime(const int &hbTime)
623 {
624

```



```

625     if (hbTime <= 0) { // zero or negative means disable the heartbeat timer
626         heartBeatTimer->stop();
627         heartBeatTime = 0;
628         heartBeat = false;
629     }
630     else {
631         heartBeatTime = hbTime;
632         if (heartBeat) { // restart the heartbeat timer if running
633             heartBeatTimer->stop();
634             heartBeatTimer->start(hbTime*1000);
635         }
636     }
637 }
638
639 // Send the heartbeat message to the ISIS server. This is a private
640 // slot connected to the timeout() signal of the heartBeatTimer()
641 // Heartbeats are prompt dispatch messages.
642
643 void ISISClient::sendHeartBeat()
644 {
645     if (!isisServerID.isEmpty())
646         send(isisServerID, "");
647 }

```